In [103…

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.colors as colors

# Define the quadratic loss function J(w, b)
# This function calculates the mean squared error for three data points:
# (150, 200), (200, 600), and (260, 500)
def calculate_loss(w, b):
    return (
        ((15*w + b - 20)**2 +
         (20*w + b - 60)**2 +
         (26*w + b - 50)**2) / 3
    )


def surface(W,B,Z):
    """
    Builds surface plots
    Input: Matrices W, B, Z
    Output matplotlib surface chart
    """
    # Set up the plot parameters
    plt.rcParams['font.size'] = 16
    # Create custom colormap
    colors_palette = [
        '#4a90e2',    # Blue
        '#f8e71c',    # Yellow
        '#ff6b6b'     # Coral/Red
    ]
    custom_cmap = colors.LinearSegmentedColormap.from_list('custom', colo

    # Create and setup the 3D plot
    fig = plt.figure(figsize=(10, 8))
    ax = fig.add_subplot(111, projection='3d')

    # Plot the surface
    surface = ax.plot_surface(W, B, Z, cmap=custom_cmap)

    # Set labels and adjust plot appearance
    ax.set_xlabel('$w$', fontsize=16)
    ax.set_ylabel('$b$', fontsize=16)
    ax.set_zlabel('$J(w,b)$', fontsize=16)
    ax.set_box_aspect(aspect=None, zoom=0.95)

    # Adjust layout and display plot
    plt.tight_layout()
    plt.show()
```

In [104…

```
# Generate parameter space for w and b
w_values = np.linspace(-5, 5, 5)
b_values = np.linspace(-10, 10, 5)
W, B = np.meshgrid(w_values, b_values)
Z = calculate_loss(W, B)

print(w_values)
```

```
print(b_values)
```

```
[-5.  -2.5  0.   2.5  5. ]
[-10.  -5.   0.   5.  10.]
```

In [105…

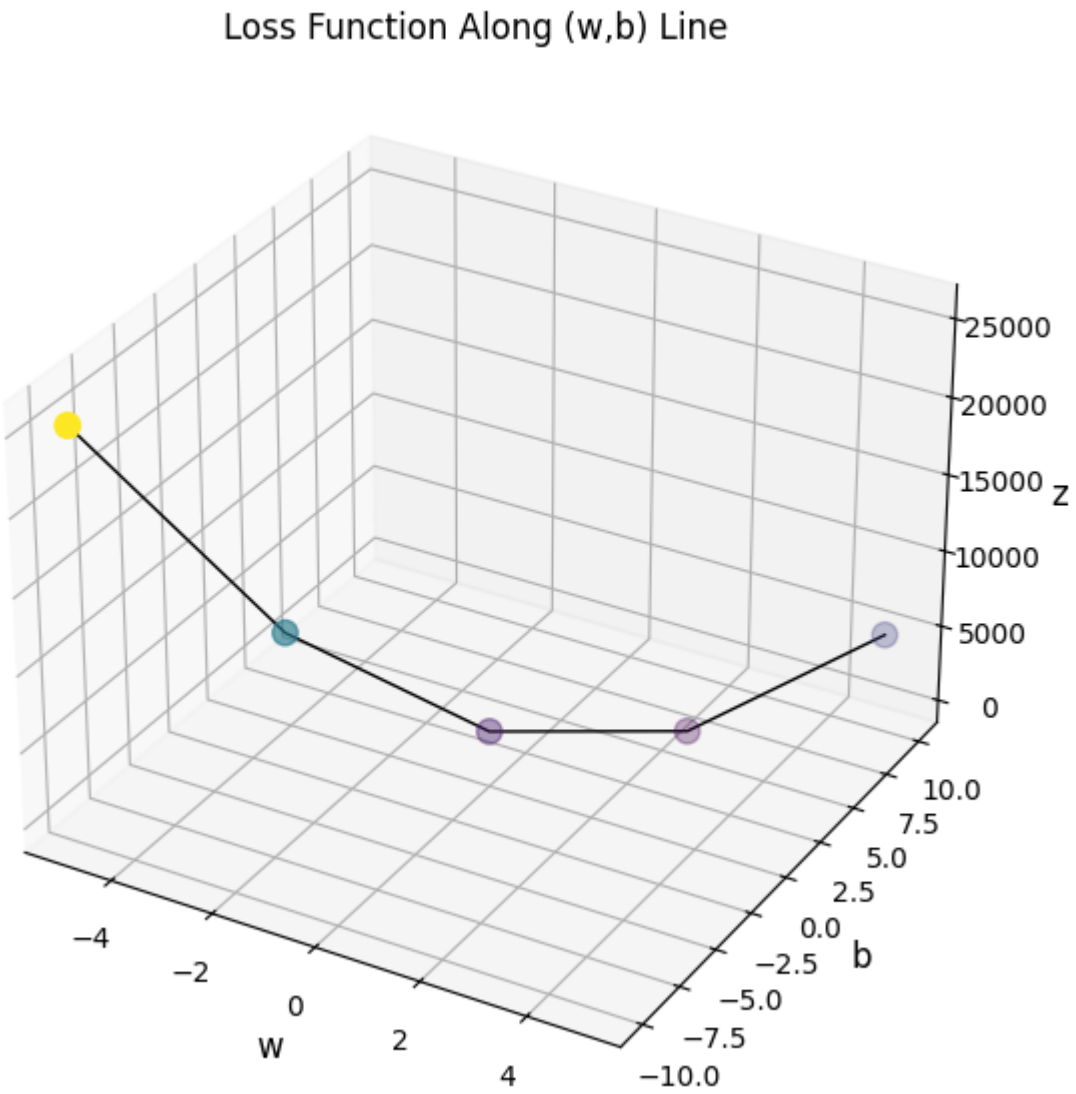
```
df = pd.DataFrame([w_values, b_values]).T
df.columns=['w', 'b']
df['z'] = df.apply(lambda x: calculate_loss(x['w'], x['b']), axis = 1)
df
```

Out[105…

	w	b	z
0	-5.0	-10.0	25341.666667
1	-2.5	-5.0	10510.416667
2	0.0	0.0	2166.666667
3	2.5	5.0	310.416667
4	5.0	10.0	4941.666667

In [106…

```
def scatter_3d (w, b, z, title=None):
    """
    Builds line plot in 3-dimentional space
    Input: vectors w, b, z
    Output: matplotlib scatter
    """
    fig = plt.figure(figsize=(8,6))
    ax = fig.add_subplot(111, projection='3d')
    ax.scatter(df['w'], df['b'], df['z'], c=df['z'], cmap='viridis', s=80)
    ax.plot(w, b, z, color='black', linewidth=1)
    ax.set_xlabel('w', fontsize=12)
    ax.set_ylabel('b',fontsize=12)
    ax.set_zlabel('z', fontsize=12)
    if title:
        ax.set_title(title, fontsize = 12)
    ax.tick_params(labelsize=10)
    plt.tight_layout()
    plt.show()
scatter_3d(df['w'], df['b'], df['z'], title='Loss Function Along (w,b) Lin
```



In [107…
```
df_w = pd.DataFrame(np.meshgrid(w_values, b_values)[0])
df_w
```

Out[107…

	0	1	2	3	4
0	-5.0	-2.5	0.0	2.5	5.0
1	-5.0	-2.5	0.0	2.5	5.0
2	-5.0	-2.5	0.0	2.5	5.0
3	-5.0	-2.5	0.0	2.5	5.0
4	-5.0	-2.5	0.0	2.5	5.0

In [108…
```
df_b = pd.DataFrame(np.meshgrid(w_values, b_values)[1])
df_b
```

Out[108…

	0	1	2	3	4
0	-10.0	-10.0	-10.0	-10.0	-10.0
1	-5.0	-5.0	-5.0	-5.0	-5.0
2	0.0	0.0	0.0	0.0	0.0
3	5.0	5.0	5.0	5.0	5.0
4	10.0	10.0	10.0	10.0	10.0

In [109…
```
calculate_loss(10,5)
```

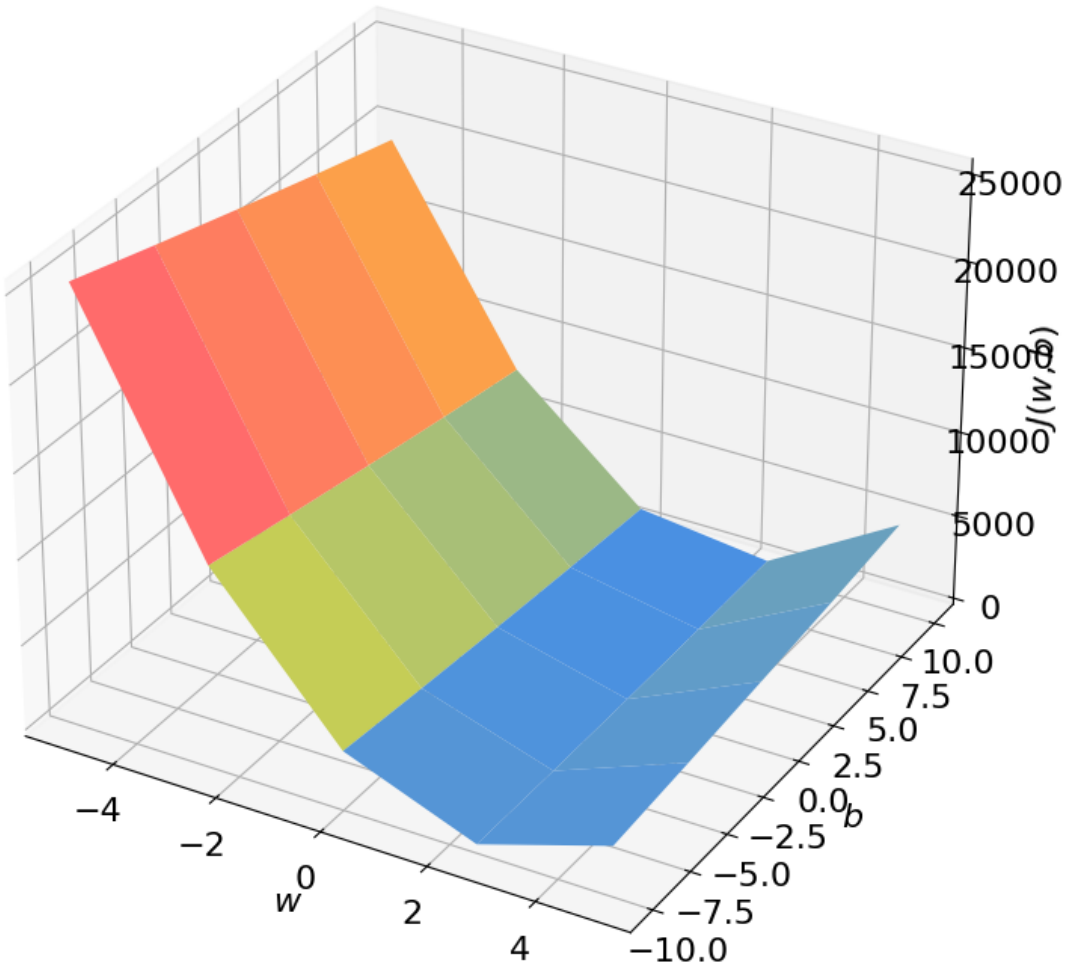
Out[109… 28491.666666666668

In [110…
```
df_z = pd.DataFrame(calculate_loss(W, B))
df_z.columns=w_values
df_z.index=b_values
df_z
```

Out[110…

	-5.0	-2.5	0.0	2.5	5.0
-10.0	25341.666667	11527.083333	3133.333333	160.416667	2608.333333
-5.0	23816.666667	10510.416667	2625.000000	160.416667	3116.666667
0.0	22341.666667	9543.750000	2166.666667	210.416667	3675.000000
5.0	20916.666667	8627.083333	1758.333333	310.416667	4283.333333
10.0	19541.666667	7760.416667	1400.000000	460.416667	4941.666667

In [111…
```
surface(W,B,Z)
```



Mathematical solution

Solving with Derivatives

Using partial differential equation:

You're using the (scaled) MSE

$$J(w,b) = \tfrac{1}{3}\Big[(15w+b-20)^2 + (20w+b-60)^2 + (26w+b-50)^2\Big].$$

Compute the gradient:

$$\begin{aligned}
\frac{\partial J}{\partial w} &= \frac{2}{3}\Big[15(15w+b-20) + 20(20w+b-60) + 26(26w+b-50)\Big] \\
&= \frac{2602}{3}\,w + \frac{122}{3}\,b - \frac{5600}{3}, \\
\frac{\partial J}{\partial b} &= \frac{2}{3}\Big[(15w+b-20) + (20w+b-60) + (26w+b-50)\Big] \\
&= \frac{122}{3}\,w + 2b - \frac{260}{3}.
\end{aligned}$$

Set them to zero (first-order optimality) and clear the thirds:

$$\begin{cases} 2602\,w + 122\,b = 5600, \\ 122\,w + 6\,b = 260. \end{cases}$$

Solve the 2×2 linear system:

$$w = \frac{235}{91} \approx 2.5824, \qquad b = -\frac{835}{91} \approx -9.1758.$$

Plugging back gives the minimum loss

$$J(w^*, b^*) = \frac{42050}{273} \approx 154.03.$$

Solving with Linear Algebra

(Equivalent linear-algebra view: with $X = \begin{bmatrix} 15 & 1 \\ 20 & 1 \\ 26 & 1 \end{bmatrix}$, $y = \begin{bmatrix} 20 \\ 60 \\ 50 \end{bmatrix}$, solve the normal equations $(X^\top X)\theta = X^\top y$ for $\theta = [w\ b]^\top$ to get the same result.)

1) Normal-equations derivation (matrix form)

For your scaled data

$x = \begin{bmatrix}15\\20\\26\end{bmatrix}$, $y = \begin{bmatrix}20\\60\\50\end{bmatrix}$

with an intercept, the design matrix is

$$X = \begin{bmatrix}15 & 1\\20 & 1\\26 & 1\end{bmatrix}, \qquad \theta = \begin{bmatrix}w\\b\end{bmatrix}.$$

Least squares minimizes $\|y - X\theta\|_2^2$. Setting the gradient to zero gives the **normal equations**:

$$X^\top X\,\theta = X^\top y.$$

Compute the pieces:

$$X^\top X = \begin{bmatrix}15^2+20^2+26^2 & 15+20+26\\15+20+26 & 3\end{bmatrix} = \begin{bmatrix}1301 & 61\\61 & 3\end{bmatrix},$$

$$X^\top y = \begin{bmatrix}15\cdot 20 + 20\cdot 60 + 26\cdot 50\\20+60+50\end{bmatrix} = \begin{bmatrix}2800\\130\end{bmatrix}.$$

Solve the 2×2 system

$$\begin{cases}1301\,w + 61\,b = 2800\\61\,w + 3\,b = 130\end{cases}$$

or equivalently

$$\theta = (X^\top X)^{-1} X^\top y.$$

The determinant is $\det = 1301\cdot 3 - 61^2 = 182$, so

$$(X^\top X)^{-1} = \frac{1}{182}\begin{bmatrix}3 & -61\\-61 & 1301\end{bmatrix}.$$

Multiply:

$$\theta = \frac{1}{182}\begin{bmatrix}3\cdot 2800 - 61\cdot 130\\-61\cdot 2800 + 1301\cdot 130\end{bmatrix} = \begin{bmatrix}\frac{235}{91}\\-\frac{835}{91}\end{bmatrix} \approx \begin{bmatrix}2.5824\\-9.1758\end{bmatrix}.$$

Minimum MSE:

$$J(\theta^*) = \frac{1}{3}\|y - X\theta^*\|_2^2 = \frac{1}{3}\left(y^\top y - \theta^{*\top} X^\top y\right) = \frac{42050}{273} \approx 154.03.$$

(The value 160.42 at $w = 2.5, b = -10$ is just the **best on your coarse grid**; the analytic optimum is slightly lower at the values above.)

2) Geometric interpretation

Columns of X span a 2-D subspace of $\mathbb{R}^3$.
The fitted vector $\hat{y} = X\theta^*$ is the **orthogonal projection** of y onto $\mathrm{col}(X)$.
Equivalently, the residual $r = y - \hat{y}$ is orthogonal to each column of X:

$$X^\top r = 0 \iff X^\top X\,\theta^* = X^\top y.$$

---

3) Handy scalar shortcut for 1 feature + intercept

With one feature, you can avoid matrices:

Let $\bar{x} = \frac{1}{N}\sum x_i,\ \bar{y} = \frac{1}{N}\sum y_i$. Then

$$w = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}, \qquad b = \bar{y} - w\,\bar{x}.$$

For your data:

- $\bar{x} = \frac{15+20+26}{3} = \frac{61}{3}$,
- $\bar{y} = \frac{20+60+50}{3} = \frac{130}{3}$,
- $\sum (x_i - \bar{x})(y_i - \bar{y}) = \frac{470}{3}$,
- $\sum (x_i - \bar{x})^2 = \frac{182}{3}$,

so $w = \frac{470/3}{182/3} = \frac{235}{91}$ and $b = \bar{y} - w\bar{x} = -\frac{835}{91}$ — the same result as the matrix method.

When to tweak

- If $X^\top X$ is singular/ill-conditioned (multicollinearity), use the **Moore–Penrose pseudoinverse** $\theta = X^+ y$ or **ridge**: $(X^\top X + \lambda I)^{-1} X^\top y$.
- The factor $1/3$ in your MSE doesn't affect θ^*; it only rescales the minimum value.